
Diabetes Risk Stratification Project

What Has Been Built So Far

A complete, from-the-ground-up walkthrough of the raw data, every file in the repository, every line of code written, and every number produced.

Covers the work from an empty folder through to a verified data split, three trained baseline models, a live results registry, and a validated rough-set engine. Nothing is assumed. Every technical term is explained the first time it is used.

Project state	Stage 0 complete, baselines recorded
Files written	9 scripts and config files
Dataset	CDC Diabetes Health Indicators (UCI #891, BRFSS 2015)
Rows after cleaning	229,474 of an original 253,680
Headline number so far	Logistic regression PR-AUC 0.4191
Number to beat it against	Prevalence baseline 0.1529

Contents

Part I The raw material — what the dataset actually is

Where the data comes from · The shape of the file · What every column means · Three properties that shape everything · The defects you inherited

Part II The workspace — how the project is laid out

The file map · What a virtual environment is · What Git is doing here

Part III File by file, line by line

config.py · bench.py · .gitignore and requirements.txt · 00_fetch_data.py · 01_stage0_split.py · data_io.py · 02_baselines.py · 03_gamma_check.py

Part IV The files your code produced

The raw CSV · dedup.csv · splits.npz · results/registry.csv

Part V Every number you have, and what it means

The integrity numbers · The baseline numbers · The rough-set verification · Your filled-in decision log

Part VI The ideas underneath

Prevalence and imbalance · Why accuracy lies · PR-AUC, ROC-AUC, Brier · Leakage · Rough sets from zero

Part VII Where you stand, and what comes next

Progress against the plan · The solo-project rescope · The next three things to do

Part I — The Raw Material

Before any code, you need to know exactly what you were handed. Almost every design decision in this project traces back to a property of this dataset, so this section is not background reading — it is the justification for everything in Part III.

1.1 Where the data comes from

The file is called **CDC Diabetes Health Indicators**, and it lives in the UCI Machine Learning Repository as dataset number 891. But UCI did not collect it. It is a cleaned-up slice of the **Behavioral Risk Factor Surveillance System** (BRFSS), which is an annual telephone survey run by the US Centers for Disease Control.

That origin matters more than anything else about the data. Picture how it was actually produced: someone's phone rings, an interviewer reads out a list of health questions, and the person answers from memory. Nobody visits a clinic. Nobody draws blood. Every single value in your 229,474 rows is a thing a person *said about themselves* over the phone in 2015.

PLAIN ENGLISH — WHAT 'SELF-REPORTED' MEANS FOR YOU

There is no blood test anywhere in this data. No HbA1c, no fasting glucose, no glucose tolerance test. The column you are trying to predict does not mean 'this person has diabetes'. It means **'this person says a doctor once told them they have diabetes'**.

Those are different things, and the gap between them is where a large part of your project's original contribution comes from. Hold onto this.

1.2 The shape of the file

The raw file is a plain CSV — a text file where each line is one person and commas separate the values. It has:

- **253,680 rows.** One row per survey respondent.
- **22 columns.** 21 of them are the questions (called *features*, or *condition attributes* in rough-set language). The 22nd is the answer you are trying to predict (called the *target*, *label*, or *decision attribute*).
- **No missing values.** Every cell has a number in it.
- **All numeric.** There is no text anywhere. Even 'yes/no' answers are stored as 1 and 0, and even things like education level are stored as numbered bands rather than words.

TRAP — 'NO MISSING VALUES' IS NOT 'NO MISSING DATA'

A real telephone survey is full of refusals, 'don't know' answers, and people who hang up. Those rows existed in the original BRFSS and were stripped out before this file reached UCI.

So the tidy dataset you downloaded is a *survivor* of a filtering process you did not see and cannot inspect. Everyone who could not or would not answer is already gone. That is a selection effect, and it is worth one honest sentence in your limitations section.

1.3 What every column means

You need this table to read any of your own results. The **Type** column matters because it decides how each variable gets handled later: binary variables are already usable by rough set theory, whereas counts like BMI have to be put into bins first.

Column	Type	Range	What it actually means
Diabetes_binary	Binary	0 / 1	The target. 1 = says a health professional told them they have diabetes or prediabetes. 0 = neither.
HighBP	Binary	0 / 1	Ever told they have high blood pressure.
HighChol	Binary	0 / 1	Ever told they have high cholesterol.
CholCheck	Binary	0 / 1	Had a cholesterol check in the last five years. Note this measures access to healthcare as much as it measures health.
BMI	Count	~12-98	Body Mass Index: weight in kg over height in metres squared. The only variable with a genuinely wide spread of values (about 80 distinct ones).
Smoker	Binary	0 / 1	Has smoked 100+ cigarettes in their <i>lifetime</i> . This is 'ever', not 'currently' — an important difference.
Stroke	Binary	0 / 1	Ever told they had a stroke.
HeartDiseaseorAttack	Binary	0 / 1	Coronary heart disease or a heart attack.
PhysActivity	Binary	0 / 1	Any physical activity in the past 30 days outside of their job.
Fruits	Binary	0 / 1	Eats fruit at least once a day.
Veggies	Binary	0 / 1	Eats vegetables at least once a day.
HvyAlcoholConsump	Binary	0 / 1	Heavy drinker: 14+ drinks a week for men, 7+ for women.
AnyHealthcare	Binary	0 / 1	Has any kind of health coverage. Key variable for your equity analysis.
NoDocbcCost	Binary	0 / 1	In the last 12 months, needed a doctor but could not afford one. Key variable for your equity analysis.
GenHlth	Ordinal	1-5	Self-rated general health, 1 = excellent to 5 = poor. Leakage-suspect.
MentHlth	Count	0-30	Days of poor mental health in the past 30.
PhysHlth	Count	0-30	Days of poor physical health in the past 30. Leakage-suspect.
DiffWalk	Binary	0 / 1	Serious difficulty walking or climbing stairs. Leakage-suspect.
Sex	Binary	0 / 1	0 = female, 1 = male.
Age	Ordinal	1-13	Five-year age bands. 1 = 18-24, 2 = 25-29, up to 13 = 80 and over. Already in bins for you.
Education	Ordinal	1-6	1 = never attended school, up to 6 = college graduate.
Income	Ordinal	1-8	Household income bands, 1 = under \$10k up to 8 = \$75k and above.

1.4 Three properties that shape everything

These three facts are the reason your project is framed the way it is. If you only remember three things from this section, make it these.

It is cross-sectional, not longitudinal

Every row is one person at one moment. Nobody was followed up a year later. There is no 'before' and no 'after'. Every feature was measured at the exact same instant as the label, so nothing in the data comes *before* the diagnosis in time.

This is why the project cannot honestly be called **early detection**. Early detection means catching disease before it shows up, which needs either a follow-up study or a real blood test that finds undiagnosed people. This data has neither. Anyone who knows the dataset can dismantle an 'early detection' claim in one sentence, which is exactly why your project is instead framed as **non-invasive risk stratification and screening triage**: given only answers a person can give over the phone, how well can you sort people into tiers deciding who should be sent for a real blood test first?

The label is systematically wrong

A 0 in the target column covers two completely different groups of people: those who genuinely do not have diabetes, and those who *do* have it but have never been diagnosed, because they never saw a doctor, could not afford one, or have no insurance. The CDC's own estimate is that roughly a fifth to a quarter of US diabetes cases are undiagnosed.

PLAIN ENGLISH — WHY THIS NOISE IS WORSE THAN RANDOM NOISE

If labels were flipped at random, the model would just be a bit worse everywhere. Annoying, harmless.

But the flipping here is **concentrated in poor and uninsured people** — and those exact characteristics are sitting in your feature matrix as AnyHealthcare, NoDocbcCost and Income.

So the model learns, quite correctly given what it was shown, that being poor and uninsured predicts *not* having diabetes. That is backwards from reality, and it means your screening tool will under-flag exactly the people who most need flagging. A tool meant to help people without lab access, that works worst on people without lab access, has inverted its own purpose. Naming that mechanism precisely is your strongest single contribution.

The classes are heavily imbalanced

Only about 14% of people in the raw file are labelled positive. The other 86% are negative. This is not a defect to be removed — it is the clinical reality, and it is what makes the problem hard. It is also the single biggest reason your choice of evaluation metric matters so much, which Part VI covers in detail.

A balanced 50/50 version of this dataset circulates widely and is popular because it makes accuracy scores look impressive. You deliberately did not use it. Removing the imbalance removes the problem.

Part II — The Workspace

Before walking through the code, here is the shape of what you built and why each piece is there.

2.1 The file map

Path	Kind	What it is for
.venv/	folder	The isolated Python installation for this project. Not committed to Git.
data/raw/...BRFSS2015.csv	data	The downloaded dataset, with columns put in a fixed order. Not committed.
data/dedup.csv	data	The dataset after duplicate rows were removed. Not committed — it can always be regenerated.
splits.npz	data	The frozen row numbers for train / validation / test, plus the data hash. Committed.
results/registry.csv	data	The append-only log of every experiment result. Committed.
config.py	code	Thread settings, locked in from the benchmark.
bench.py	code	One-off benchmark that measured how fast your CPU trains a network.
00_fetch_data.py	code	Downloads the dataset from UCI and writes the raw CSV.
01_stage0_split.py	code	Counts duplicates, removes them, splits the data, computes the hash.
data_io.py	code	Shared helper. Loads the data, checks the hash, logs results.
02_baselines.py	code	Trains logistic regression, a decision tree, and a random forest.
03_gamma_check.py	code	Verifies the rough-set dependency-degree function against a hand-worked example.
.gitignore	config	Tells Git which files to ignore.
requirements.txt	config	The exact list of installed packages.

2.2 What a virtual environment is, and why yours exists

Your computer has one system-wide Python. If every project installed its packages there, two projects needing different versions of the same package would fight, and you would have no way of telling a collaborator exactly what to install.

A **virtual environment** solves this. `python -m venv .venv` creates a folder called `.venv` containing a private copy of Python and a private, empty package folder. When you 'activate' it, the word `python` in your terminal starts pointing at that private copy instead of the system one. Everything you install goes in there and affects nothing else on your machine.

TRAP — THE PROMPT LIES (YOU HIT THIS ONE)

Your terminal showed `(.venv)` at the start of the line, which normally means the environment is active. But when you typed `python 02_baselines.py` it ran
`AppData/Local/Programs/Python/Python311/python.exe` — the system Python, not yours.

The prefix is only cosmetic text. What actually matters is which `python.exe` the shell finds first. Always verify with:

`Get-Command python | Select-Object Source`

It must end in `diabetes-project/.venv/Scripts/python.exe`. If it does not, packages will appear to be missing and you will waste an hour chasing a code bug that is not there.

2.3 What Git is doing here

Git records snapshots of your project. Each snapshot is a **commit**, identified by a short code like **d0c2062**. This gives you three things that matter for a research project rather than just for software:

- **An undo button that never expires.** Any file can be restored to any past state.
- **A record of when each number was produced.** Your results registry stores the commit code alongside every metric, so in week nine you can go back and see the exact code that generated any row.
- **A way to share.** Anyone can clone the repository and reproduce your work exactly.

The `.gitignore` file lists things Git should not record. Yours excludes `.venv/` (huge, and rebuildable from `requirements.txt`), `data/` (large, and rebuildable by rerunning the fetch script), `__pycache__/` (auto-generated) and `figures/`. Everything else — all your code, the frozen splits, the results — is tracked.

WHY DATA/ IS DELIBERATELY NOT COMMITTED

Git is bad at large binary-ish files and would bloat your repository. More importantly, you do not need to: `00_fetch_data.py` plus `01_stage0_split.py` regenerate both CSVs byte-for-byte from scratch. The *recipe* is committed, so the *ingredients* do not need to be.

But `splits.npz` **is** committed, because it contains a random choice. Re-running the split with the same seed gives the same answer, but committing it removes any doubt at all.

Part III — File by File, Line by Line

This is the core of the document. For each file: what problem it solves, the code itself, an explanation of every meaningful line, and what it produced when you ran it.

3.1 config.py — locking in the thread settings

The smallest file in the project, and one that has to be imported *before* anything else in any script that trains a model.

```
# config.py
import os
N_WORKERS = 4
THREADS_PER_WORKER = 4
os.environ["OMP_NUM_THREADS"] = str(THREADS_PER_WORKER)
os.environ["MKL_NUM_THREADS"] = str(THREADS_PER_WORKER)
```

Your laptop's CPU (an Intel Core Ultra 5 125H) is a **hybrid** design: it has 4 fast 'performance' cores, 8 slower 'efficiency' cores, and 2 slower-still low-power cores — 18 logical threads in total. Normally you would want to use all of them.

You do not, and the benchmark in the next section shows why. These two environment variables tell the numerical libraries underneath PyTorch (OpenMP and Intel MKL) to use only 4 threads.

WHY THIS MUST BE IMPORTED FIRST

OpenMP and MKL create their pools of worker threads at the moment they are first loaded, which happens when you `import torch`. Once created, the pool size cannot be changed.

So `import config` has to come *above* `import torch` in every file. If it comes after, the setting is read too late and silently does nothing — no error, just slower code.

3.2 bench.py — measuring your own machine

A throwaway script with a permanent payoff. It trains a fake neural network on fake data of exactly the right size, at several different thread counts, and times each one. You run it once, write down the answer, and never run it again.

```

import os
os.environ["OMP_NUM_THREADS"] = "8"
os.environ["MKL_NUM_THREADS"] = "8"
import time, torch, torch.nn as nn
torch.manual_seed(0)

N, D, BATCH, EPOCHS = 253_680, 21, 1024, 5
X = torch.randn(N, D)          # fake features, right shape
y = (torch.rand(N) < 0.14).float() # fake labels at ~14% positive

def make_model():
    return nn.Sequential(nn.Linear(D, 64), nn.BatchNorm1d(64), nn.ReLU(), nn.Dropout(0.3),
                        nn.Linear(64, 32), nn.BatchNorm1d(32), nn.ReLU(), nn.Dropout(0.2),
                        nn.Linear(32, 1))

def run_epoch(model, opt, crit):
    perm = torch.randperm(N)          # shuffle the row order
    for i in range(0, N, BATCH):      # walk through in chunks of 1024
        idx = perm[i:i + BATCH]
        opt.zero_grad()
        crit(model(X[idx]).squeeze(1), y[idx]).backward()
        opt.step()

for threads in [4, 6, 8, 10, 14]:
    torch.set_num_threads(threads)
    model = make_model(); opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
    crit = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(6.0))
    run_epoch(model, opt, crit)      # untimed warm-up
    t0 = time.perf_counter()
    for _ in range(EPOCHS):
        run_epoch(model, opt, crit)
    print(f"threads={threads:2d}  {(time.perf_counter()-t0)/EPOCHS:.3f} sec/epoch")

```

What the pieces do

- `torch.randn(N, D)` makes a table of random numbers with 253,680 rows and 21 columns — the same shape as the real data. Timing does not care whether the numbers are real.
- `run_epoch` does one full pass over the data. It shuffles the rows, then takes them 1024 at a time. For each chunk it pushes the data through the network, measures the error, calls `.backward()` to work out how each weight contributed to that error, and `opt.step()` to nudge every weight slightly. One pass = one **epoch**.
- The first, untimed **warm-up** pass is deliberate. The very first pass pays one-off costs — memory allocation, starting the thread pool — that would unfairly penalise whichever setting happened to be measured first.

The result, and why it is surprising

PyTorch threads	Seconds per epoch
4	0.541 ← fastest
6	0.693
8	0.633
10	0.910
14	0.786

PLAIN ENGLISH — WHY 4 THREADS BEATS 14

Training synchronises at every single weight update — about 248 times per epoch here. Each update can only finish when its **slowest** thread finishes.

Ask for more than 4 threads and the extra work gets placed on the slow efficiency cores. Now every update waits for them. You have not added capacity; you have added a slower critical path.

The wobble in the middle (6 slower than 8, 10 slower than 14) is Windows scheduler noise — it does not reliably decide which core type gets which thread. Read the trend, not the wobble.

The practical consequence, recorded in **config.py**: when you eventually run many experiments at once, run **4 separate processes with 4 threads each**, not one process with 16. Measured, that roughly doubles total throughput. Doing the opposite — 4 processes each asking for 14 threads — is nearly three times *slower* than just running the jobs one after another.

3.3 .gitignore and requirements.txt

```
__pycache__/  
.venv/  
data/  
figures/
```

.gitignore — four lines, each excluding a folder from version control.

You originally had *.csv here, which would have been a serious mistake: it would have silently excluded results/registry.csv, so your entire results log would never have been committed and would have been lost the moment anything went wrong. Changing it to data/ keeps the large data files out while letting the small, precious results file through.

requirements.txt is generated by `pip freeze > requirements.txt` and lists every installed package with its exact version. Anyone (including future you on a different machine) can recreate your environment exactly with `pip install -r requirements.txt`. Regenerate it whenever you install something new — you had to after adding ucimlrepo.

3.4 00_fetch_data.py — getting the data, once

This script downloads the dataset from UCI, assembles it into a single table with columns in a fixed order, checks it, and writes it to disk. It runs once. Nothing else in the project ever touches the network.

```

from ucimlrepo import fetch_ucirepo
import pandas as pd
from pathlib import Path

TARGET = "Diabetes_binary"
RAW = Path("data/raw/diabetes_binary_health_indicators_BRFSS2015.csv")

ds = fetch_ucirepo(id=891)
print("target cols:", ds.data.targets.columns.tolist())

df = pd.concat([ds.data.features, ds.data.targets], axis=1)

# canonical column order so everyone's data hash matches
feats = sorted(c for c in df.columns if c != TARGET)
df = df[feats + [TARGET]]

assert df.shape == (253680, 22), df.shape
assert df.isna().sum().sum() == 0

RAW.parent.mkdir(parents=True, exist_ok=True)
df.to_csv(RAW, index=False)
print("wrote", RAW, df.shape)

```

Line by line

- `fetch_ucirepo(id=891)` calls UCI's servers and returns a bundle object. The features and the target come back as two *separate* tables, in `ds.data.features` and `ds.data.targets`.
- `pd.concat(..., axis=1)` glues them side by side into one table of 22 columns. `axis=1` means 'join left-to-right' rather than stacking them on top of each other.
- `sorted(...)` then reordering is the important line. It puts the 21 feature columns in alphabetical order and the target last, *every single time*, no matter what order UCI happens to send them in.
- The two `assert` statements are self-checks. If the shape is not exactly 253,680 by 22, or if any cell is empty, the script stops immediately with an error rather than quietly writing a broken file.
- `mkdir(parents=True, exist_ok=True)` creates the `data/raw/` folders if they do not exist. `to_csv(..., index=False)` writes the file without adding a pointless extra column of row numbers.

WHY THE FIXED COLUMN ORDER MATTERS SO MUCH

The next script computes a **hash** — a short fingerprint of the data. That fingerprint changes if the columns are in a different order, even though the data is identical.

So without the sort line, two people running the same code could get two different hashes and conclude their data was different when it was not. Even working alone this protects you: re-download in three weeks, get a different column order, and suddenly nothing matches.

WHAT WENT WRONG WHEN YOU FIRST RAN THIS

Your first version was the example snippet copied straight from the UCI website. It fetched the data and printed the metadata table — the one that scrolled past showing each variable's name, role and 'has missing values: no'.

But it never combined the tables and never wrote a file. That is why `Get-ChildItem data/raw` said the path did not exist. The script ran perfectly and did nothing useful, which is the most common kind of bug in this sort of work.

What it produced

target cols: [Diabetes_binary] confirming the column name, then wrote data/raw/...csv (253680, 22), and an 11.8 MB file on disk. Smaller than you might expect for a quarter of a million rows, because most values are a single character.

3.5 01_stage0_split.py — the integrity step

The most consequential script you have written. It measures the data's defects, removes duplicates, splits the rows into three groups, and fingerprints the result. Everything downstream depends on it, which is why it runs once and its output is frozen.

```
import pandas as pd, numpy as np, hashlib
from pathlib import Path
from sklearn.model_selection import train_test_split

TARGET = "Diabetes_binary"
RAW = Path("data/raw/diabetes_binary_health_indicators_BRFSS2015.csv")

df = pd.read_csv(RAW)
feats = [c for c in df.columns if c != TARGET]

print("rows:", len(df))
print("prevalence:", round(df[TARGET].mean(), 4))
print("exact dupes:", df.duplicated().sum())
print("feature-identical:", df[feats].duplicated().sum())

g = df.groupby(feats)[TARGET].nunique()
print("contradictory groups:", int((g > 1).sum()))

df = df.drop_duplicates().reset_index(drop=True)
print("rows after dedup:", len(df))
print("prevalence after dedup:", round(df[TARGET].mean(), 4))

h = hashlib.md5(pd.util.hash_pandas_object(df, index=False).values).hexdigest()
print("DATA HASH:", h)

idx = np.arange(len(df))
tr, tmp = train_test_split(idx, test_size=0.30, stratify=df[TARGET], random_state=42)
va, te = train_test_split(tmp, test_size=0.50, stratify=df[TARGET].iloc[tmp], random_state=42)

np.savez("splits.npz", train=tr, val=va, test=te, data_hash=h)
df.to_csv("data/dedup.csv", index=False)
print("splits:", len(tr), len(va), len(te))
```

The four counting lines

df[TARGET].mean() gives the **prevalence**. Because the target is only ever 0 or 1, the average *is* the fraction of 1s. Yours came out at 0.1393, so 13.93% of people are labelled positive.

df.duplicated().sum() counts **exact duplicate rows** — rows where all 22 values, features and label, match a row that appeared earlier. .duplicated() returns a true/false value per row, and summing true/false values counts the trues. You got 24,206.

df[feats].duplicated().sum() does the same but ignores the label, so it counts **feature-identical** rows: people who gave the same 21 answers, regardless of whether they have diabetes. You got 25,772.

The groupby line is the subtle one. It gathers every group of people who gave identical answers, and nunique() asks how many *different* labels appear inside each group. If a group has 1, everyone in it agrees. If it has 2, the group contains both diabetics and non-diabetics with identical answers. (g > 1).sum() counts those **contradictory groups**. You got 1,566.

THESE NUMBERS ARE INTERNALLY CONSISTENT — A GOOD SIGN

25,772 feature-identical minus 24,206 exact duplicates = **1,566**, exactly the contradictory-group count. That is not a coincidence, and it confirms the three measurements agree with each other. Those 1,566 rows are people whose answers match someone else's but whose outcome differs — and deduplication correctly leaves them alone, because they are genuinely different data points.

Deduplication, and why it must come before splitting

`drop_duplicates()` keeps the first occurrence of each repeated row and discards the rest. `reset_index(drop=True)` then renumbers the rows from 0 with no gaps, and `drop=True` throws away the old numbering instead of keeping it as a column.

TRAP — THE SILENT SCORE INFLATION THIS PREVENTS

If you split into train and test *first*, some test rows would be byte-identical to training rows. The model would have already seen them and memorised the answer.

Your reported test score would then be part memorisation. **Nothing would error.** The number would simply be too good, and you would have no way of knowing. This is the most common silent failure in published work on this dataset.

The hash

`hashlib.md5(...).hexdigest()` reduces the entire table to a 32-character fingerprint:

fc3774b0bdf6c0a16e80c7e518cd787d. Change a single value anywhere in 229,474 rows and the fingerprint changes completely. It is a cheap way to prove two datasets are identical without comparing them cell by cell.

`pd.util.hash_pandas_object(df, index=False)` first turns each row into a number; `index=False` makes it ignore row numbering so only the contents matter.

The split

Two calls, because the tool only splits in two. The first takes 30% off the end; the second cuts that 30% in half, giving 70 / 15 / 15.

- `stratify=df[TARGET])` forces every piece to contain the same proportion of diabetics as the whole. Without it, a random split could by chance put more positive cases in one group than another, and your metrics would jitter between runs for reasons having nothing to do with your model.
- `random_state=42` fixes the randomness. Re-run it a year from now and you get the identical split. The number itself is arbitrary; what matters is that it never changes.
- `np.savez` saves the three lists of row numbers plus the hash into one file, `splits.npz`. Note it stores **which rows**, not the rows themselves.

THE FROZEN TEST SET IS NOT A SUGGESTION

The test group of 34,422 rows must be touched **exactly once**, at the very end of the project, to produce the final numbers for your paper.

Every decision along the way — which features, which architecture, which threshold — is made using the *validation* group. The moment you evaluate on test, change something, and evaluate again, the test set has quietly become a second validation set and your final number is optimistic. Put this rule in your README.

What it produced

Printed	Value	Meaning
rows	253,680	As expected.
prevalence	0.1393	13.93% positive before cleaning.
exact dupes	24,206	Identical across all 22 columns.
feature-identical	25,772	Identical across the 21 features.
contradictory groups	1,566	Same answers, different outcome. Drives Stage 2.
rows after dedup	229,474	24,206 rows removed.
prevalence after dedup	0.1529	Rose by 1.4 points — see Part V.
DATA HASH	fc3774b0...	The fingerprint every script now checks.
splits	160,631 / 34,421 / 34,422	Train / validation / test.

3.6 data_io.py — the shared helper

Not a script you run. It is a **module**: a file of functions other scripts import. It exists so that loading the data, checking the fingerprint, and recording a result happen the same way everywhere, written once.

```
import pandas as pd, numpy as np, hashlib
from pathlib import Path
import subprocess, datetime, csv, os

TARGET = "Diabetes_binary"
DEDUP = Path("data/dedup.csv")
SPLITS = Path("splits.npz")
LEAKAGE_SUSPECT = ["GenHlth", "DiffWalk", "PhysHlth"]
DATA_HASH = None

def load():
    """Returns (df, train_idx, val_idx, test_idx). Asserts the data hash."""
    df = pd.read_csv(DEDUP)
    s = np.load(SPLITS, allow_pickle=True)
    h = hashlib.md5(pd.util.hash_pandas_object(df, index=False).values).hexdigest()
    expected = str(s["data_hash"])
    assert h == expected, f"DATA HASH MISMATCH\n got {h}\n want {expected}"
    globals()["DATA_HASH"] = h
    return df, s["train"], s["val"], s["test"]

def features(df, drop_leakage=False):
    cols = [c for c in df.columns if c != TARGET]
    if drop_leakage:
        cols = [c for c in cols if c not in LEAKAGE_SUSPECT]
    return cols
```

What load() is really for

It recomputes the fingerprint of the data on disk and compares it against the one saved inside splits.npz. If they differ, the assert stops the script immediately with a loud message.

This catches a specific, nasty failure: someone re-downloads the CSV, or skips deduplication, or edits a value — and then produces a week of results on subtly different data without noticing. With the check, that becomes impossible. Every script that loads data now proves it is looking at exactly the 229,474 rows the split was built for.

THE BUG YOU HAD HERE

You first placed `globals()["DATA_HASH"] = h` at the **top** of `load()`, before `h` had been calculated. Python would have raised `UnboundLocalError` — you cannot use a variable before the line that creates it.

Moving it below the `assert` fixed it. The line's job is to make the hash visible to other scripts so they can record it alongside every result.

`features()` returns the list of column names to use, optionally dropping the three leakage-suspect ones. Putting that list in *one place* means the definition of 'leakage-free' can never drift between scripts.

The results registry functions

```
REGISTRY = Path("results/registry.csv")
FIELDS = ["timestamp", "git_commit", "data_hash", "machine", "script",
          "feature_set", "n_features", "model", "params", "seed", "split",
          "pr_auc", "roc_auc", "brier", "prevalence", "notes"]

def git_commit():
    try:
        return subprocess.check_output(
            ["git", "rev-parse", "--short", "HEAD"], text=True).strip()
    except Exception:
        return "nogit"

def log_result(**kw):
    """Append one row to the append-only results registry."""
    REGISTRY.parent.mkdir(exist_ok=True)
    row = {k: "" for k in FIELDS}
    row.update({
        "timestamp": datetime.datetime.now().isoformat(timespec="seconds"),
        "git_commit": git_commit(),
        "machine": os.environ.get("COMPUTERNAME", "?"),
    })
    row.update(kw)
    unknown = set(kw) - set(FIELDS)
    assert not unknown, f"unknown registry fields: {unknown}"
    new = not REGISTRY.exists()
    with REGISTRY.open("a", newline="") as f:
        w = csv.DictWriter(f, fieldnames=FIELDS)
        if new:
            w.writeheader()
        w.writerow(row)
```

- `git_commit()` shells out to Git and asks for the current commit's short code. The `try/except` means that if Git is unavailable it returns `nogit` rather than crashing your experiment.
- `log_result(**kw)` — the `**kw` means 'accept any named arguments and collect them into a dictionary'. That is why you can call it with `pr_auc=0.4191`, `model="logistic regression"` and so on.
- It starts with every field blank, fills in the three automatic ones (timestamp, commit, machine name), then overlays whatever you passed.
- The `unknown` assert catches typos. Write `pr_uac=` by mistake and it stops immediately, instead of silently discarding the number.
- `open("a")` means **append** mode: it adds to the end of the file and never overwrites. The header row is written only if the file did not already exist.

WHY APPEND-ONLY, AND WHY NEVER EDIT IT

In week nine you will write your paper from this file. If results can be overwritten, a re-run silently destroys the number you cited last week.

Duplicates are fine and expected — run the same script twice and you get two rows. The timestamp and commit code tell them apart. Do not add de-duplication logic; the one time it would trigger is the one time you wanted both rows.

3.7 02_baselines.py — the numbers everything must beat

This trains three simple, well-understood models on the real data and records how they do. Their scores become the bar every fancier model has to clear. If your neural network cannot beat logistic regression, that is a finding, and you need to know it early rather than in week eight.

```

import numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (average_precision_score, roc_auc_score,
                             brier_score_loss, accuracy_score)
from data_io import load, features, log_result, TARGET
import data_io

df, tr, va, te = load()

for drop_leak in [False, True]:
    cols = features(df, drop_leakage=drop_leak)
    Xtr, ytr = df.loc[tr, cols], df.loc[tr, TARGET]
    Xva, yva = df.loc[va, cols], df.loc[va, TARGET]

    prev = yva.mean()
    tag = "leakage-free" if drop_leak else "all features"
    print(f"\n=== {tag} ({len(cols)} features) ===")
    print(f"PR-AUC baseline (prevalence): {prev:.4f}")
    print(f"majority-class accuracy:      {1 - prev:.4f}    <- the accuracy trap")

    models = {
        "logistic regression": make_pipeline(
            StandardScaler(),
            LogisticRegression(max_iter=2000, class_weight="balanced")),
        "decision tree (d=6)": DecisionTreeClassifier(
            max_depth=6, class_weight="balanced", random_state=42),
        "random forest": RandomForestClassifier(
            n_estimators=300, min_samples_leaf=20, class_weight="balanced",
            n_jobs=4, random_state=42),
    }

    for name, m in models.items():
        m.fit(Xtr, ytr)
        p = m.predict_proba(Xva)[:, 1]
        pr = average_precision_score(yva, p)
        roc = roc_auc_score(yva, p)
        br = brier_score_loss(yva, p)
        print(f"{name:22s} PR-AUC {pr:.4f} ROC-AUC {roc:.4f} Brier {br:.4f}")
        log_result(data_hash=data_io.DATA_HASH, script="02_baselines.py",
                   feature_set="leakage-free" if drop_leak else "all_21",
                   n_features=len(cols), model=name, params=str(m.get_params()),
                   seed=42, split="val", pr_auc=round(pr, 4),
                   roc_auc=round(roc, 4), brier=round(br, 4),
                   prevalence=round(prev, 4),
                   notes="class_weight=balanced, uncalibrated")

```

How the script is organised

The outer loop runs the entire experiment twice — once with all 21 features, once with the three leakage-suspect ones removed. That single `for drop_leak in [False, True]` line is doing real scientific work, and Part V explains what it found.

`df.loc[tr, cols]` selects rows and columns at once: the rows listed in `tr` (your training row numbers), and the columns in `cols`. `X` is the input, `y` is the answer, following the usual convention.

NOTICE WHAT IS MISSING: THE TEST SET

te is loaded and then never used. That is deliberate and correct. Everything you have measured so far is on the **validation** group.

This is the discipline that keeps your final number honest.

The three models

- **Logistic regression** draws one straight boundary through the data. Simple, fast, interpretable, and on tabular survey data surprisingly hard to beat.
- **Decision tree** asks a sequence of yes/no questions. `max_depth=6` limits it to six questions deep so it does not simply memorise the training data.
- **Random forest** builds 300 different trees on random pieces of the data and averages them. `min_samples_leaf=20` stops any branch from being based on fewer than 20 people; `n_jobs=4` uses your four fast cores, per the benchmark.

`make_pipeline(StandardScaler(), LogisticRegression(...))` chains two steps. The scaler rescales every column to a comparable range, which logistic regression needs because BMI (values around 12–98) would otherwise dominate a 0/1 column purely by being numerically larger. Trees do not care about scale, so they get no scaler.

`class_weight="balanced"` tells each model to treat the rare positive cases as more important, roughly in proportion to how rare they are. Without it, all three would learn that guessing 'no diabetes' every time is a fine strategy.

`m.fit(Xtr, ytr)` does the learning. `m.predict_proba(Xva)[: , 1]` then asks for predictions on the validation set. It returns two columns — probability of 0 and probability of 1 — and `[: , 1]` keeps the second, giving each person a risk score between 0 and 1.

A CAVEAT ABOUT THESE BRIER SCORES

`class_weight="balanced"` deliberately distorts the predicted probabilities upward. That is fine for ranking people, but it means the Brier scores here (all around 0.18) look bad and are **not** comparable to a properly calibrated model later.

Do not put these in your calibration table. They are ranking baselines, not calibration baselines.

3.8 03_gamma_check.py — proving the rough-set engine works

The rough-set part of your project has no reliable off-the-shelf Python library, so you are writing it yourself. Before running home-made maths on 229,474 rows, you check it against six rows whose answer you worked out by hand.

```
import pandas as pd, numpy as np

toy = pd.DataFrame({
    "HighBP": [1, 1, 1, 0, 0, 0],
    "Smoker": [1, 1, 0, 0, 1, 1],
    "Diabetes": [1, 1, 0, 0, 1, 0],
})

def gamma_vprs(df, B, target, beta=1.0):
    if not B:
        return 0.0
    g = df.groupby(list(B))[target].agg(['size', 'sum'])
    p_pos = g['sum'] / g['size'] # positive rate inside each group
    purity = np.maximum(p_pos, 1 - p_pos) # how one-sided the group is
    return g['size'][purity >= beta].sum() / len(df)

B = ["HighBP", "Smoker"]
print(round(gamma_vprs(toy, B, "Diabetes", 1.0), 4)) # expect 0.6667
print(round(gamma_vprs(toy, B, "Diabetes", 0.6), 4)) # expect 0.6667
print(round(gamma_vprs(toy, B, "Diabetes", 0.5), 4)) # expect 1.0
```

Working the toy example by hand

Six people, two questions. Group them by their answers:

Answers (HighBP, Smoker)	People	Diabetes labels	Verdict
(1, 1)	u1, u2	1, 1	All positive — pure
(1, 0)	u3	0	All negative — pure
(0, 0)	u4	0	All negative — pure
(0, 1)	u5, u6	1, 0	Mixed — contradictory

Four of the six people sit in groups where everyone agrees, so you can classify them with certainty from these two questions alone. Two people (u5 and u6) gave identical answers but had different outcomes — these two questions simply cannot tell them apart.

So the **dependency degree**, written γ , is 4 out of 6 = **0.6667**. That is the fraction of the population you can classify with certainty. It runs from 0 (the questions tell you nothing) to 1 (the questions fully determine the answer).

What β does

Classical rough set theory demands a group be 100% one class before it counts as certain. On real survey data that is far too strict — a group of 100 people where 92 are diabetic gets thrown away entirely because of 8 exceptions.

Variable Precision Rough Sets relaxes this with a threshold β . A group counts as certain if it is at least β one-sided. $\beta = 1.0$ recovers the strict classical version; lower values tolerate more noise.

- **$\beta = 1.0 \rightarrow 0.6667$** . The mixed group (50/50) is rejected. Four people counted.
- **$\beta = 0.6 \rightarrow 0.6667$** . The mixed group is 50% pure, which is below 0.6, so it is still rejected. Same answer.
- **$\beta = 0.5 \rightarrow 1.0$** . Now 50% clears the bar, the mixed group is admitted, and all six people count.

All three printed exactly as predicted, in about one second. The engine is verified.

WHY THIS USES GROUPBY INSTEAD OF THE TEXTBOOK METHOD

Textbooks compute reducts using a **discernibility matrix**: a grid recording, for every *pair* of people, which attributes tell them apart. At 229,474 people that is over 52 billion cells. It cannot be done on any machine you have access to.

But the dependency degree only needs to know, for each distinct combination of answers, how the labels are distributed inside it. That is precisely what groupby gives you. It runs in seconds and uses tens of megabytes.

Any tutorial showing you a discernibility matrix is quietly assuming a dataset of a few hundred rows.

Part IV — The Files Your Code Produced

Four artefacts now exist that did not before. Knowing what is inside each one matters, because two of them are the foundation everything else is built on.

4.1 data/raw/...BRFSS2015.csv

11.8 MB. The dataset exactly as UCI holds it, but with columns forced into a fixed order. Not committed to Git — it is regenerated by running `00_fetch_data.py`. Treat it as read-only from now on. Nothing should ever modify this file.

4.2 data/dedup.csv

229,474 rows, the raw file with 24,206 exact duplicate rows removed and the row numbering reset from zero. This is the dataset your project actually uses. Also not committed, because it is fully reproducible from the raw file.

The renumbering matters more than it sounds: the row numbers stored in `splits.npz` refer to positions in *this* file. Regenerate it differently and the splits would point at the wrong people — which is exactly what the hash check exists to catch.

4.3 splits.npz

A small compressed file holding four things. `.npz` is NumPy's format for storing several arrays together.

Key	Contents	Size
train	Row numbers used for learning	160,631
val	Row numbers for all decisions and tuning	34,421
test	Row numbers touched exactly once, at the end	34,422
data_hash	The fingerprint <code>fc3774b0bdf6c0a16e80c7e518cd787d</code>	1 value

This file is committed to Git, and it is the single most important artefact you have produced. It is what makes every future result comparable to every past one. Do not regenerate it. Do not delete it.

4.4 results/registry.csv

Seven lines right now: one header and six results, one per model per feature set. Sixteen columns capturing not just the metrics but the full context needed to reproduce them.

Column group	Fields	Why it is there
Provenance	timestamp, git_commit, data_hash, machine	Lets you go back to the exact code and data that produced any row, months later.
Configuration	script, feature_set, n_features, model, params, seed, split	Distinguishes two rows that would otherwise look identical. The <code>split</code> field records whether a number came from validation or test — which you will be very glad of in week nine.
Metrics	pr_auc, roc_auc, brier, prevalence	The actual results, plus the baseline they should be read against.
Free text	notes	Anything unusual about the run.

When you write the paper, this file *is* your results section. Without it you would be re-running experiments to recover numbers you had already computed and forgotten.

Part V — Every Number You Have

You have produced roughly twenty numbers so far. Several of them are more interesting than expected, and two of them change the plan.

5.1 The integrity numbers, and what they tell you

Prevalence rose from 0.1393 to 0.1529

This is the first genuinely informative thing your code found. Removing 24,206 duplicate rows pushed the positive rate up by 1.36 percentage points, which means **the duplicates were disproportionately negative**.

That makes sense on reflection. Duplicates happen when two people give identical answers to all 21 questions. With mostly yes/no answers and a quarter of a million people, collisions are inevitable — and they collide most often among the large, healthy majority, whose answers are more alike. Sick people have more varied profiles.

PRACTICAL CONSEQUENCE — UPDATE YOUR BASELINE EVERYWHERE

0.1529 is now your PR-AUC baseline, not the 0.14 quoted in the design document. Every result in your paper is measured against this number. Correct it wherever it appears.

It also means the majority-class accuracy trap sits at **0.8471**: a model that predicts 'no diabetes' for all 34,421 validation people scores 84.71% accuracy while catching zero diabetics.

Only 1,566 contradictory groups

This one changes your Stage 2 plan. The design document warned that contradictions would be 'substantial' and would destroy classical rough set analysis. At 1,566 out of 25,772 feature-identical rows, the contradiction rate is much lower than anticipated.

The reason is that BMI has around 80 distinct values, and MentHlth and PhysHlth have 31 each. Combined, they make almost every row's answer profile unique. Groups of size one are trivially pure.

WHAT THIS PREDICTS ABOUT YOUR ROUGH SET RESULTS

On the raw, un-binned features you should expect **failure mode A**: γ close to 1.0 for almost any subset of attributes, and a greedy search stopping after three or four attributes with a 'reduct' that is an artefact of high cardinality rather than a discovery about diabetes.

Failure mode B — where the boundary region swallows everything and γ collapses toward zero — will only appear *after* you bin BMI into five categories and the health-days variables into three.

So you still get both failure modes, which is what makes the motivation section of your paper work. But they arrive in the opposite order to the one the design document assumed, and you should say so.

5.2 The baseline numbers

Feature set	Model	PR-AUC	ROC-AUC	Brier
All 21	Logistic regression	0.4191	0.8126	0.1838
All 21	Decision tree (depth 6)	0.3961	0.7982	0.1898
All 21	Random forest	0.4426	0.8176	0.1760
Leakage-free 18	Logistic regression	0.3906	0.7940	0.1914
Leakage-free 18	Decision tree (depth 6)	0.3683	0.7770	0.1973
Leakage-free 18	Random forest	0.4121	0.7987	0.1837

All on the validation set. PR-AUC baseline (prevalence) = 0.1529.

Reading these

- **Logistic regression at 0.4191 is the number to beat.** Against a baseline of 0.1529 that is 2.7 times better than random guessing. There is real, learnable signal in this data.
- **Random forest already beats it** at 0.4426. This previews something your design document warns about: on tabular data with mostly binary features, tree ensembles are extremely strong. Expect gradient boosting to beat your neural network too. That is not a failure — report it honestly and state the network's real role in the pipeline.
- **The decision tree is worst**, which is expected. A single tree limited to six questions is a deliberately weak model, included to make the comparison honest.

The leakage result — your first real finding

Removing GenHlth, DiffWalk and PhysHlth costs logistic regression 0.0285 PR-AUC, from 0.4191 down to 0.3906. That is about 7% in relative terms.

But the fairer way to read it is as a fraction of the *lift above baseline*, since 0.1529 is free. Lift goes from 0.2662 to 0.2377, a loss of about 11%.

WHY THIS IS GOOD NEWS FOR YOUR PROJECT

Those three variables were suspected of being **consequences** of diabetes rather than causes of it — someone with neuropathy and fatigue rates their general health as poor and struggles to climb stairs *because* they are already diabetic.

A model leaning heavily on them would be partly detecting 'this person is already unwell', which is a much easier and much less useful task.

Your measurement shows the effect is real but **bounded**: roughly a tenth of the usable signal. That means you can run the leakage-free 18-feature configuration as your headline without giving up much — a more defensible position than the design document anticipated, and one that almost no published work on this dataset reports at all.

5.3 The rough-set verification

Three printed values: 0.6667, 0.6667, 1.0 — matching the hand-worked example exactly. Runtime about one second on six rows. Your `gamma_vprs` function is correct and ready to run on real data.

5.4 Your decision log, filled in

Appendix C of the design document asks you to record these as you go. Copy this table into the repository.

Item	Value	Where it came from
Rows, raw	253,680	01_stage0_split.py
Prevalence, raw	0.1393	Before deduplication
Exact duplicate rows	24,206	<code>df.duplicated().sum()</code>
Feature-identical rows	25,772	Ignoring the label
Contradictory groups	1,566	Drives Stage 2
Rows after dedup	229,474	24,206 removed
Class prevalence	0.1529	The PR-AUC baseline
Data hash	fc3774b0bdf6c0a16e80c7e518cd787d	Asserted by every script

Item	Value	Where it came from
Split seed / proportions	42 / 70-15-15	Stratified on the label
Split sizes	160,631 / 34,421 / 34,422	Train / val / test
LR baseline PR-AUC	0.4191	All 21 features, val
LR leakage-free PR-AUC	0.3906	18 features, val
Majority-class accuracy	0.8471	For the accuracy-trap slide
sec/epoch, 4 threads	0.541	Machine A, batch 1024
Worker configuration	4 processes × 4 threads	2.1× net throughput
Discretisation chosen	not yet decided	Stage 1
γ without discretisation	not yet measured	Failure mode A evidence
γ with discretisation, $\beta=1$	not yet measured	Failure mode B evidence
Chosen β	not yet decided	From the sensitivity curve
Clinical operating point	not yet decided	e.g. precision = 0.30
Final test evaluation date	must happen exactly once	Week 8

Part VI — The Ideas Underneath

Every concept your code relies on, explained from scratch. If any line in Part III felt like it was doing something for unclear reasons, the reason is here.

6.1 Prevalence and class imbalance

Prevalence is the fraction of people who actually have the condition — 0.1529 in your cleaned data.

Class imbalance is the fact that this is far from 50/50: for every diabetic there are about 5.5 non-diabetics.

Imbalance makes the problem hard because a model can score well on naive measures by essentially ignoring the rare class. It also makes several standard tools behave badly, which is why so much of your project design is organised around it.

6.2 Why accuracy lies

THE 84.71% THAT MEANS NOTHING

A model that outputs 'no diabetes' for every single person — a model with no intelligence whatsoever, that could be written in one line — scores **84.71% accuracy** on your validation set.

It identifies **zero** diabetics. It is completely useless. And it beats many accuracy figures published on this dataset.

This is why accuracy can never be your headline metric here. Compute it if you like for comparison, but always print the majority-class baseline immediately next to it so a reader can see what the number is worth.

6.3 The metrics that do work

Precision and recall

Precision: of the people you flagged as at-risk, what fraction actually have diabetes. **Recall** (also called sensitivity): of the people who actually have diabetes, what fraction did you catch.

They trade off against each other. Flag everybody and your recall is perfect but your precision drops to the base rate. Flag only the single most obvious case and precision is perfect but recall is almost zero. Any useful model lives somewhere in between, and where exactly is a clinical decision, not a mathematical one.

PR-AUC — your primary metric

Short for **area under the precision-recall curve**, also called average precision. It summarises the precision/recall trade-off across every possible threshold into one number between 0 and 1.

The crucial property: **its baseline is the prevalence**. A useless random model scores 0.1529. So your logistic regression at 0.4191 is 2.7 times better than nothing. This is why PR-AUC is the right headline metric — it focuses entirely on the minority class you care about, and it comes with a built-in point of comparison.

ROC-AUC — report it, do not lead with it

The probability that the model gives a randomly chosen diabetic a higher score than a randomly chosen non-diabetic. Its baseline is always 0.5, regardless of imbalance.

That last property is exactly the problem. Under heavy imbalance it looks flatteringly high, because the enormous negative class makes the false-positive rate appear small no matter what you do. Your 0.8126 sounds much more impressive than your 0.4191, and the 0.4191 is the more honest number. Report ROC-AUC for comparison with published work; do not build your argument on it.

Brier score — measuring calibration

The average squared difference between the predicted probability and the actual 0 or 1 outcome. Lower is better. It measures **calibration**: whether, among all the people you assigned a 30% risk, roughly 30% really do have diabetes.

WHY CALIBRATION IS STRUCTURAL IN YOUR PROJECT, NOT OPTIONAL

In an ordinary classification paper, calibration is a nice-to-have. Here it holds up the whole design.

Your Stage 4 fuzzy layer takes the predicted probability and attaches words to it — 'Low', 'Moderate', 'High' risk. If the model is miscalibrated, so that an output of 0.4 actually corresponds to a real 12% rate, then the word 'Moderate' is meaningless and the entire fuzzy layer is built on sand.

This is why calibration appears throughout your plan rather than as an afterthought.

6.4 Leakage

Leakage is when information that would not really be available at prediction time sneaks into the model's inputs, making test performance look far better than real-world performance. It comes in two shapes, and your code guards against both.

- **Consequence features.** A feature that is actually an *effect* of the outcome rather than a cause. GenHlth, DiffWalk and PhysHlth are the suspects here, and section 5.2 quantifies what they are worth.
- **Partition leakage.** The same information appearing in both training and test. Your duplicate removal before splitting is exactly this defence.

A third form will matter later: fitting any preprocessing step on all the data before splitting. Scalers, binning schemes and reduct computation must all be fitted on the training rows only, then applied to validation and test. Otherwise information about the test answers has flowed backwards into your preprocessing.

6.5 Stratification, seeds and hashes

Stratified splitting forces each of your three groups to contain the same proportion of positives as the whole dataset. With a 15% minority class this matters a great deal; without it, metrics jitter between runs for reasons that have nothing to do with your model.

A **random seed** is a starting number for the random-number generator. Fixing it at 42 makes every 'random' choice repeatable. Run the split next year and you get exactly the same 160,631 training rows.

A **hash** is a short fingerprint computed from data. Any change to the data, however small, produces a completely different fingerprint. It lets you prove two datasets are identical without comparing them value by value.

6.6 Rough sets from zero

Rough Set Theory was introduced by Zdzisław Pawlak in 1982. Its central idea is very simple: **if two objects have identical values on the attributes you are looking at, you cannot tell them apart**. Everything else follows from that.

Term	What it means in your data
Decision table	Just your table. Rows are people, the 21 features are <i>condition attributes</i> , and the diabetes label is the <i>decision attribute</i> .
Indiscernibility	Pick some subset B of the questions. Two people are indiscernible with respect to B if they gave identical answers to every question in B.

Term	What it means in your data
Granule	A group of people who gave identical answers. In your code, one row of the groupby result.
Lower approximation	All the granules that are entirely diabetic. These are people you can classify with <i>certainty</i> — everyone who answered like them also has diabetes.
Upper approximation	All granules containing at least one diabetic. These are people who <i>might</i> be diabetic.
Boundary region	Upper minus lower: the mixed granules. Your 1,566 contradictory groups live here. The attributes are simply insufficient to decide.
Positive region	Everyone in a pure granule of any class — all the people you can classify with certainty.
Dependency degree (γ)	The size of the positive region divided by the total. The fraction of people classifiable with certainty. Exactly what your <code>gamma_vprs</code> function returns.
Reduct	A minimal subset of questions that keeps γ at the same value as using all of them. Remove any one question from a reduct and γ drops. This is your feature-selection output.
Core	The questions appearing in every possible reduct — the indispensable ones.

WHY ROUGH SETS RATHER THAN AN ORDINARY FEATURE SELECTOR

Unlike most methods, rough set theory needs no external parameter, no assumption about distributions, and no threshold you have to choose. It does not assume a straight-line relationship the way LASSO does, and it does not make you pick 'top k' the way mutual information does. It derives the answer from the structure of the data alone.

It also produces IF-THEN rules as a by-product, which feed directly into your Stage 4 fuzzy rule base. That is what makes rough sets and fuzzy logic *one system* in your project rather than two unrelated stages bolted together.

Part VII — Where You Stand

7.1 Progress against the plan

The design document's Appendix B lists ten things to do in the first 48 hours. You have done eight of them, in an afternoon.

Appendix B item	Status
Download the dataset; confirm 253,680 rows and 22 columns	Done
Run every structural check and record the numbers	Done
Count contradictory groups	Done — 1,566
Deduplicate, split 70/15/15, save splits.npz, record the hash	Done
Create the Git repo, .gitignore, empty results registry	Done
Verify gamma_vprs against the toy example	Done — 0.6667 / 0.6667 / 1.0
Train logistic regression on all 21 features; record PR-AUC	Done — 0.4191
Compute and note the majority-class accuracy	Done — 0.8471
Run the CUDA check on Machine B	Not done — only needed if you keep NSGA-II
Agree both interface contracts in writing	Outstanding — still worth doing for yourself

7.2 The rescope, now that you are working alone

The design document is written for four people over ten weeks. Alone, you have roughly a quarter of the labour and the same calendar. Its own closing advice becomes binding: five or six differentiators executed carefully beat twelve attempted superficially.

Keep

- **The five non-negotiables:** deduplication, the frozen test set, multiple seeds, PR-AUC as headline metric, the leakage-free comparison. You already have four of them in place.
- **Rough sets: both failure modes plus the β sensitivity sweep.** This is the theoretical spine of the project, and it is cheap — the engine is already written and verified.
- **The bias and equity audit.** Highest contribution per hour in the entire document. It is subgroup tables plus a retraining loop, and it is the most distinctive thing you will have.
- **The fuzzy layer, Designs A and B.** The soft-computing requirement of the course, and hand-implementing Mamdani inference is around 150 lines of NumPy.

Cut or defer

- **NSGA-II multi-objective optimisation.** The big one. It is the only genuinely compute-hungry stage and the least essential. Cutting it also removes any need for Machine B, so the whole project runs on the laptop.
- **Fuzzy-rough feature selection.** Keep VPRS instead.
- **The genetic-algorithm reduct search.**
- **The prevalence-relative VPRS extension** and the three-class prediabetes experiment.

WHAT YOU WOULD STILL BE DELIVERING

Honest reframing, deduplication, quantified leakage, imbalanced data as primary, PR-AUC plus calibration, both rough-set failure modes demonstrated numerically, a β sensitivity curve, a four-way imbalance ablation, a genuinely multi-input fuzzy layer, and the equity analysis.

That is **ten of the twelve rows** in the design document's differentiation table, minus only the two that cost the most compute. It is comfortably above coursework level.

7.3 The next three things to do

1. Run γ on the real data, un-binned

Five lines using what you already have. Load the data via `data_io`, call `gamma_vprs` with all 21 raw features. Expect γ close to 1.0. That single number is your failure-mode-A evidence and the opening of your rough-set motivation section.

2. Decide the discretisation scheme and measure γ again

Bin BMI into the five WHO categories, and `MentHlth` and `PhysHlth` into `{0, 1-13, 14-30}`. Re-measure γ . Expect it to collapse. That is your failure-mode-B evidence, and together the two numbers motivate everything that follows.

FIT THE BINNING ON TRAINING ROWS ONLY

If you use a supervised binning method such as MDLP, it looks at the labels to choose its cutpoints. Fit it on the whole dataset and label information from your test set has flowed into your preprocessing.

The clinically-grounded cutpoints avoid this entirely, since they come from WHO guidelines rather than from your data — which is a small extra argument in their favour.

3. Add LightGBM to the baselines

About a minute of work and a minute of runtime. It is the realistic strong baseline for tabular data, and having it early tells you what your neural network is actually competing against. Given that the random forest is already at 0.4426, you want to know this number before you invest a week in the MLP.

One habit worth keeping. You put the headline number in your commit message — *"results registry + baselines (LR PR-AUC 0.4191)"*. Keep doing that. When you are hunting in week nine for the point at which a number changed, the commit log reads like a lab notebook.